AIFFEL AI RESEARCHER 18 · SP 02

DeepSeek-V3 + R1 심화 학습 기록

효율의 아키텍처와 추론의 창발
— 두 기술보고서의 되먹임 구조 읽기

작성 (기록이)	추병곤
이끄미	김택훈
살리미	김정민 · 유비 · 김택훈
일자	2026-08-25
캔버스	tldraw `3xFjUwVXIjXIGxoGA9GtL`

목차

오늘의 역할

오늘의 목표

루브릭 역산 (무엇을 "증명"해야 하는가)

오늘의 시간표

오늘의 학습내용 / 토의

0. 이 노드의 좌표 — 왜 V3와 R1을 한 세트로 읽는가

1. DeepSeek-V3 아키텍처 — 세 개의 축

1-1. MLA (Multi-head Latent Attention) — KV 캐시라는 병목

1-2. DeepSeekMoE + 보조손실 없는 부하분산 (Auxiliary-Loss-Free Load Balancing)

1-3. MTP (Multi-Token Prediction)

2. 학습 인프라 — "557만 달러"라는 숫자의 해부

비용 분해 (H800 GPU-hour, $2/hour 가정)

DualPipe — 파이프라인 버블 죽이기

FP8 혼합정밀 학습 — 세계 최초 초대형 규모 검증

기타 메모리 최적화

3. 사전학습 데이터와 컨텍스트 확장

4. DeepSeek-R1 — 추론은 가르치는 것인가, 자라는 것인가

4-1. R1-Zero: SFT를 완전히 건너뛴 순수 RL

4-2. GRPO (Group Relative Policy Optimization) — 반드시 손으로 유도해볼 것

4-3. R1의 4단계 파이프라인

4-4. 증류(Distillation) vs 직접 RL — 논문의 가장 중요한 실험

4-5. 실패한 시도 (논문이 공개한 네거티브 결과) — 이 절이 가장 값지다

5. 두 논문을 관통하는 네 개의 명제 (토의 정리)

6. 자주 나오는 오개념 체크리스트

오늘의 실험

Peer Review 30분 전 — 회고 정리

잘 된 점

아쉬운 점

✓ 해결해낸 어려웠던 개념

▲ 아직 더 공부해야 할 어려운 개념

코드 완성 후 — 확장 실험 제안

◆ E1. KV 캐시 계산기 + mini-MLA (난이도 ★★)

◆ E2. 보조손실 없는 부하분산 시뮬레이션 (난이도 ★★)

◆ E3. MTP λ ablation (난이도 ★★★)

◆ E4. GRPO 그룹 크기 G 스윕 (난이도 ★★★)

◆ E5. "Aha Moment" 정량 탐지 (난이도 ★★)

◆ E6. 보상 항목 ablation (난이도 ★★★)

◆ E7. 정렬세(Alignment Tax) 정량화 (난이도 ★★★)

◆ E8. 루브릭 확장 실험

다음 노드 연결 지점

다음 회차 준비 (숙제)

오늘의 역할

역할	담당
이끄미	김택훈
살리미	김정민, 유비, 김택훈
기록이	추병곤

오늘의 목표

1. 오늘의 주제에 작성한 내용을 **"아키텍처 효율(V3)" / "추론 창발(R1)" / "둘의 되먹임 구조"** 세 층위로 명확히 구분한다.
2. 각 장별 **[목차 · 목표 · 루브릭]** 을 대조하여 학습·프로젝트 방향성을 설정한다.
3. 학습의 **가지(branch)** 에 해당 — 본 줄기(Transformer/RLHF)에서 뻗어나온 최신 효율화·추론 계열의 분기점으로 위치시킨다.

루브릭 역산 (무엇을 "증명"해야 하는가)

루브릭 항목	증명해야 할 것	산출물
구조 이해	MLA·MoE·MTP를 **왜** 그렇게 설계했는지 제약조건으로부터 설명	KV 캐시 계산표 + 도식
구현 능력	최소 구현(mini-MLA, mini-GRPO)이 돌아감	노트북 셀
실험 설계	하이퍼파라미터 1개를 스윕하고 해석	ablation 표 + 그래프
비판적 독해	논문이 **말하지 않은 것**을 지적	토의 노트

오늘의 시간표

- 1일차: **15:30** 토의
- 2일차: **15:30** 토의
- (Peer Review 30분 전 회고 항목 사전 정리 — 하단 참조)

오늘의 학습내용 / 토의

0. 이 노드의 좌표 — 왜 V3와 R1을 한 세트로 읽는가

두 모델은 별개의 릴리스가 아니라 **하나의 순환 구조**다.

```
DeepSeek-V3-Base
      |
      |  순수 RL (SFT 0건)
      v
DeepSeek-R1-Zero        ... "추론은 스스로 자라는가"의 증명
      |
      |  콜드스타트 SFT + 4단계 파이프라인
      v
DeepSeek-R1
      |
      |  추론 능력 distill 되먹임
      v
DeepSeek-V3 (Chat) 완성
```

- **V3 = 효율(Efficiency)의 극한** — "같은 성능을 얼마나 싸게 만들 수 있는가"
- **R1 = 창발(Emergence)의 증명** — "지도 없이 추론이 스스로 자라는가"
- 그리고 R1이 만든 추론 데이터가 다시 V3의 post-training으로 흘러들어간다.

> ◆ **토의 프레임**: V3가 없었으면 R1은 불가능했다(RL은 비싸다 → base가 싸야 한다). R1이 없었으면 V3 Chat은 평범했다. **효율은 스케일의 반대가 아니라 스케일의 전제조건이다.**

---

1. DeepSeek-V3 아키텍처 — 세 개의 축

1-1. MLA (Multi-head Latent Attention) — KV 캐시라는 병목

문제 정의부터. 추론 시 메모리를 잡아먹는 것은 파라미터가 아니라 **KV 캐시**다.

```
KV 캐시 크기 = 2 × L(층수) × n_h(헤드수) × d_h(헤드차원) × seq_len × precision
```

61층 × 128헤드 × 128차원 = 토큰당 **약 500K 원소**. 128K 컨텍스트면 캐시만으로 수백 GB.

기존 해법과 그 대가

방식	캐시량	대가
MHA	100%	없음
GQA	1/g	표현력 손실
MQA	1/n_h	품질 저하 뚜렷
MLA	**~1/14**	**거의 없음 (MHA 이상 보고)**

핵심 아이디어 — 저랭크 결합 압축(Low-Rank Joint Compression)

```
c_t^KV = W^DKV · h_t          # 잠재 벡터로 다운프로젝션 (d_c = 512)
k_t^C  = W^UK  · c_t^KV       # 필요할 때 복원
v_t^C  = W^UV  · c_t^KV
```

캐시에 저장하는 것은 k, v가 아니라 **c_t^KV 하나**다.

흡수(Absorption) 트릭 — 여기가 시험에 나온다

추론 시 `W^UK`는 쿼리 쪽 `W^UQ`에 **행렬곱으로 미리 흡수**시킬 수 있고, `W^UV`는 출력 투영 `W^O`에 흡수시킬 수 있다. 즉 복원 연산이 아예 사라진다. → 압축했는데 연산이 늘지 않는다.

Decoupled RoPE — 가장 자주 틀리는 지점

RoPE는 위치에 따라 회전 행렬을 곱한다. 그런데 회전 행렬이 `W^UK`와 `c_t^KV` 사이에 끼어들면 **교환법칙이 깨져 흡수가 불가능**해진다. 그래서 DeepSeek는 키를 두 갈래로 나눈다.

```
k_t = [ k_t^C (압축 경로, 위치정보 없음) ; k_t^R (RoPE 전용 경로, 모든 헤드가 공유) ]
```

`d_h^R = 64`짜리 작은 RoPE 전용 키만 따로 캐시. 캐시 총량 = **(d_c + d_h^R) × L = (512 + 64) × 61**.

V3 실제 수치

항목	값
층수	61 (앞 3층은 dense FFN)
hidden dim	7168
어텐션 헤드	128 × 헤드당 128
KV 압축 차원 d_c	512
Query 압축 차원 d_c'	1536
RoPE 전용 차원 d_h^R	64

> ◆ **한 줄 요약**: MLA는 "KV를 줄인 것"이 아니라 **"KV를 잠재공간에 접어 넣고, 펴는 비용을 가중치에 미리 접어 넣은 것"**.

1-2. DeepSeekMoE + 보조손실 없는 부하분산(Auxiliary-Loss-Free Load Balancing)

구조

- 공유 전문가(shared expert) **1개** — 모든 토큰이 통과, 공통 지식 담당
- 라우팅 전문가(routed expert) **256개** 중 **8개** 활성화
- 전문가 중간차원 2048 (가늘고 많게 = fine-grained segmentation)
- **총 671B / 토큰당 활성 37B** (약 5.5%)
- **Node-limited routing**: 한 토큰은 최대 **4개 노드**로만 전송 → 크로스노드 통신량 상한 고정
- **토큰 드롭 없음** (학습·추론 모두)

왜 보조손실을 버렸는가 — 오늘의 핵심 토의거리

기존 MoE는 전문가 부하를 맞추려고 auxiliary loss를 더한다. 그런데 이 손실의 그래디언트는 **언어 모델링 목적과 무관한 방향**으로 파라미터를 밀어낸다(interference gradient). 균형은 얻고 성능은 잃는다.

DeepSeek의 해법:

```
선택 기준:  argTopK( s_i + b_i )      ← 바이어스는 "선택"에만 개입
게이팅 값:  s_i                        ← 실제 가중치는 원본 그대로
```

b_i는 학습되지 않는다. 매 스텝 끝에 규칙으로 갱신된다.

```
if 전문가 i 과부하:  b_i ← b_i − γ
if 전문가 i 저부하:  b_i ← b_i + γ
```

γ(bias update speed)는 14.3T 토큰까지 0.001, 이후 500B는 0으로 고정(수렴 안정화). 그리고 **시퀀스 단위 보조손실은 α=0.0001**로 아주 약하게만 남긴다(한 문장이 한 전문가로 쏠리는 극단만 방지).

> ◆ **토의 명제**: b_i 갱신은 미분 불가능한 **제어(control) 신호**다. 즉 DeepSeek은 "부하분산"을 학습 문제에서 빼내 **피드백 제어 문제로 재정의**했다. 손실함수에 모든 것을 욱여넣는 관성에 대한 반례.

1-3. MTP (Multi-Token Prediction)

구조 (Meta 방식과의 차이가 포인트)

- Meta 2024: 여러 헤드가 **병렬**로 t+1…t+n 예측 → 인과 사슬 끊김

- DeepSeek V3: MTP 모듈을 **순차(sequential)** 로 쌓아 **완전한 인과 사슬 유지**

```
Depth k 모듈:
  h'_i^k = M_k · [ RMSNorm(h_i^{k-1}) ; RMSNorm(Emb(t_{i+k})) ]
  → Transformer 블록 1개 → 공유 출력헤드 → t_{i+k+1} 예측
```

임베딩 레이어와 출력 헤드는 **메인 모델과 공유** (메모리 절약).

손실 가중치 λ 스케줄: 첫 10T 토큰 **0.3** → 나머지 4.8T **0.1**

두 가지 효용

1. **학습 신호 밀도 증가** — 토큰 하나가 여러 예측에 기여, 표현이 "미래를 계획"하게 됨
2. **추론 시 speculative decoding** — 모듈을 버려도 되고, 남겨두면 2번째 토큰 수용률 **85~90%**, 생성 속도(TPS) **약 1.8배**

> ■ 시험 포인트: **MTP는 추론 시 선택적으로 폐기 가능하다.** 즉 학습 보조 장치이면서 동시에 추론 가속 장치라는 이중성.

---

2. 학습 인프라 — "557만 달러"라는 숫자의 해부

비용 분해 (H800 GPU-hour, $2/hour 가정)

단계	GPU-hours	비용
사전학습	2,664K	$5.328M
컨텍스트 확장	119K	$0.238M
Post-training	5K	$0.01M
합계	**2,788K**	**$5.576M**

> ▲ **비판적 독해**: 이것은 **최종 런 1회의 GPU 대여료**다. 실패한 실험, ablation, 데이터 파이프라인 구축, 인건비, R1 학습 비는 **전부 제외**되어 있다. 논문도 이를 명시한다. "600만 달러로 GPT-4를 이겼다"는 언론 서사는 이 각주를 지운 것.

DualPipe — 파이프라인 버블 죽이기

크로스노드 MoE는 **all-to-all 통신**이 필연이다. H800은 H100 대비 NVLink 대역폭이 제한된다 → 통신이 곧 병목.

- forward/backward 청크를 **{어텐션, all-to-all 디스패치, MLP, all-to-all 컴바인}** 4요소로 쪼갬

- 양방향(bidirectional) 스케줄로 **연산과 통신을 거의 완전 중첩**
- 결과: **텐서 병렬(TP) 없이** PP16 + EP64 + ZeRO-1 DP로 학습

> ◆ 하드웨어 제약(수출규제 H800)이 소프트웨어 혁신을 강제한 사례. **"제약이 아키텍처를 만든다"** — 오늘 토의의 1번 명제.

FP8 혼합정밀 학습 — 세계 최초 초대형 규모 검증

- 전 구간 **E4M3** 사용 (E5M2 혼용 안 함)
- **세밀 양자화(fine-grained quantization)**
- 활성값: **타일 단위 1×128**
- 가중치: **블록 단위 128×128**
- → 이상치(outlier) 하나가 텐서 전체 스케일을 망치는 문제 차단
- **누산 정밀도 승격**: Tensor Core의 FP8 누산은 14비트 수준 → **128개 원소마다 CUDA 코어로 옮겨 FP32 누산**

> ▲ **흔한 오해 정정**: "V3는 FP8로 학습했다" ≠ "전부 FP8이다". 마스터 가중치·옵티마이저 상태·임베딩·출력헤드·MoE 게이팅·정규화·어텐션 일부는 **BF16/FP32 유지**. 상대오차 손실 **0.25% 미만**으로 보고.

기타 메모리 최적화

- RMSNorm · MLA up-projection **재계산(recompute)**
- EMA 파라미터는 **CPU 메모리**에 비동기 보관
- MTP 임베딩/헤드 **물리적 공유**

---

3. 사전학습 데이터와 컨텍스트 확장

항목	내용
토큰	**14.8T** (고품질·다양성 강화, 수학/코드 비중 증대)
토크나이저	BBPE, vocab **128K**
FIM	Prefix-Suffix-Middle(PSM), 비율 **0.1**
LR 스케줄	워밍업 → 2.2e-4 상수 유지 → 코사인 감쇠 → 계단식 저LR 마무리
배치	3072 → 15360 점진 증가

컨텍스트 확장 — YaRN 2단계

```
4K ——(1000 스텝)——> 32K ——(1000 스텝)——> 128K
```

NIAH(Needle-In-A-Haystack) 전 구간에서 안정적 검색 성능 유지.

---

4. DeepSeek-R1 — 추론은 가르치는 것인가, 자라는 것인가

4-1. R1-Zero: SFT를 완전히 건너뛴 순수 RL

이것이 논문의 진짜 주장이다. **DeepSeek-V3-Base에 지도학습 없이 곧바로 RL만 적용.**

보상 설계 — 의도적으로 원시적

보상	내용
정확도 보상	수학은 최종 답 정규식 매칭, 코드는 컴파일러/테스트케이스 통과 여부
형식 보상	사고 과정을 `<think>...</think>` 안에 넣었는가

신경망 보상모델(Neural RM)을 쓰지 않았다. 이유: 대규모 RL에서 **reward hacking**이 발생하고, RM 재학습이 파이프라인을 복잡하게 만들기 때문.

결과

지표	RL 전	RL 후
AIME 2024 pass@1	15.6%	**71.0%**
AIME cons@64 (다수결)	—	**86.7%**

→ o1-0912 수준. **인간이 만든 추론 예시 데이터 0건으로.**

자기진화(Self-Evolution)

학습이 진행될수록 모델의 **응답 길이가 자발적으로 증가**했다. 아무도 "더 길게 생각해"라고 시키지 않았다. 모델이 스스로 **test-time compute를 더 할당**하기로 결정한 것. 그 과정에서 **반성(reflection), 재검증, 대안 탐색**이 창발.

"Aha Moment"

중간 체크포인트에서 모델이 문제를 풀다 말고 스스로 멈추고 초기 접근을 재평가하는 순간이 관측됨. 논문 저자들이 "아하 모먼트"라 명명.

> ◆ **철학적 함의**: 지능적 행동을 **설계**하지 않고, 그것이 나타날 **조건(탐색 공간 + 정확한 보상)**만 마련했다. 이것은 미학에서 말하는 생성(becoming) 의 문제와 구조적으로 닮아 있다 — 형상을 부여하는 것이 아니라 형상이 발생할 장(場)을 조성하는 것.

한계

- 가독성 나쁨 (사람이 읽기 힘든 사고 흐름)
- **언어 혼용** (영어/중국어가 한 문단에 섞임)
- → 그래서 R1이 필요했다.

4-2. GRPO (Group Relative Policy Optimization) — 반드시 손으로 유도해볼 것

PPO의 문제: 크리틱(가치 모델)이 정책 모델과 **비슷한 크기**로 하나 더 필요 → 메모리 2배. 게다가 LLM은 **마지막 토큰에서만 보상**이 나오는데 토큰별 가치를 추정하는 건 구조적으로 어렵다.

GRPO의 해법: 크리틱을 없애고, **같은 질문에 대한 G개 샘플 그룹 내부의 상대 비교**로 베이스라인을 대체.

```
1) 질문 q에 대해 정책으로 G개 응답 {o_1 ... o_G} 샘플링
2) 각각에 보상 {r_1 ... r_G} 부여
3) 그룹 정규화 우위:
        A_i = (r_i − mean(r)) / std(r)
4) 클리핑된 목적함수 + KL 페널티
        J = E[ min( ρ_i A_i , clip(ρ_i, 1−ε, 1+ε) A_i ) − β·D_KL(π_θ ‖ π_ref) ]
        단, ρ_i = π_θ(o_i|q) / π_θ_old(o_i|q)
5) KL은 비편향 추정량(k3):
        D_KL = π_ref/π_θ − log(π_ref/π_θ) − 1     (항상 ≥ 0)
```

KL 항의 위치가 중요: PPO는 보상에 KL을 섞지만, GRPO는 **손실에 직접** 더한다.

> ◆ **토의 포인트 (약점 지적)** - 그룹의 응답이 **전부 정답이거나 전부 오답이면 std=0, A_i=0** → 그래디언트가 통째로 사라짐. 샘플 낭비. - std 나눗셈은 **난이도 편향**을 만든다(쉬운 문제의 소수 오답에 과도한 가중). 후속 연구(Dr.GRPO 등)에서 std 제거 논의가 나온 배경. - G를 키우면 분산은 줄지만 **비용이 선형 증가**.

4-3. R1의 4단계 파이프라인

> ① 콜드스타트 SFT → ② 추론 RL → ③ 리젝션 샘플링 + SFT → ④ 전방위 RL

① 콜드스타트 (Cold Start) - 수천 건의 **긴 CoT** 데이터로 V3-Base를 가볍게 SFT - 가독성 템플릿 강제: `|special_token| 추론과정 |special_token| 요약` - 목적: RL 초기의 불안정한 탐색 구간을 건너뛰고 **수렴 가속**

② **추론 지향 RL** - R1-Zero와 동일한 규칙 보상 + **언어 일관성 보상**(CoT 내 목표 언어 단어 비율) - ▲ **핵심 관찰**: 언어 일관성 보상은 **성능을 약간 떨어뜨린다**. 그럼에도 채택했다 — 인간 선호도가 개선되므로. - → **정렬세(alignment tax)를 논문이 스스로 명시한 드문 사례.** 오늘 토의 필수 항목.

③ **리젝션 샘플링 + SFT** - 추론 데이터 **약 600K** (RL 체크포인트에서 샘플링 후 필터링) - 비추론 데이터 **약 200K** (글쓰기, 사실QA, 번역, 자기인식 등 — V3 파이프라인 재활용) - 합계 **800K** 로 2에폭 SFT - → 여기서 모델이 "추론 특화"에서 **"범용"** 으로 전환

④ **전방위 RL (All-Scenario)** - 추론 영역: 규칙 기반 보상 유지 - 일반 영역: 선호 기반 보상모델 - **유용성(helpfulness)은 최종 요약부만 평가, 무해성(harmlessness)은 사고과정 포함 전체 평가** ← 설계 디테일

최종 성적

벤치마크	R1
AIME 2024 (pass@1)	79.8%
MATH-500	97.3%
Codeforces Elo	~2029 (상위 96.3%)
GPQA Diamond	71.5%
MMLU	90.8%
SWE-bench Verified	49.2%

4-4. 증류(Distillation) vs 직접 RL — 논문의 가장 중요한 실험

R1의 800K 샘플로 **Qwen2.5·Llama3 계열(1.5B / 7B / 8B / 14B / 32B / 70B)** 을 SFT만으로 증류.

결정적 대조 실험 (32B 동일 조건)

모델	AIME 2024	MATH-500	LiveCodeBench
Qwen-32B에 **직접 RL** (10K+ 스텝)	47.0	91.6	40.2
R1-Distill-Qwen-32B (SFT만)	**72.6**	**94.3**	**57.2**

해석 — 두 가지 층위로 읽자

1. **경제적 결론**: 소형 모델에 막대한 RL을 돌리는 것보다, 큰 모델의 추론 패턴을 증류하는 편이 압도적으로 싸고 강하다.
2. **인식론적 결론**: RL은 **없는 능력을 만들지 못한다.** RL은 base 모델이 이미 잠재적으로 가진 능력을 **탐색을 통해 발현(elicit)** 시킬 뿐이다. 32B base의 잠재력 천장이 곧 직접 RL의 천장이었다. 그리고 그 천장을 넘는 방법은 **더 큰 모델의 궤적을 보여주는 것**이었다.

또한 R1-Distill-Qwen-**1.5B**가 수학에서 GPT-4o·Claude-3.5-Sonnet을 상회 — **추론 능력이 파라미터 수와 분리 가능함**을 시사.

4-5. 실패한 시도 (논문이 공개한 네거티브 결과) — 이 절이 가장 값지다

시도	실패 이유
PRM (Process Reward Model)	① 일반 추론에서 "단계"를 정의하는 것 자체가 어려움 ② 중간 단계 정답 판정 자동화 곤란, 수동 라벨링은 확장 불가 ③ 대규모 RL에서 **reward hacking** 발생 ④ RM 재학습 비용이 파이프라인 전체를 무겁게 함
MCTS (몬테카를로 트리 탐색)	① 체스와 달리 토큰 생성은 **탐색 공간이 지수적으로 폭발** ② 노드 확장 상한을 두면 국소최적에 빠짐 ③ 세밀한 가치 모델 학습이 본질적으로 어려움 → 자기개선 선순환 실패

> ◆ **연구 태도 토의**: 실패를 논문에 싣는 것은 흔치 않다. 이것이 "무엇이 통했는가"보다 후속 연구에 더 큰 가치를 준다. — 우리 팀 실험 기록에도 실패 로그를 남길 것.

---

5. 두 논문을 관통하는 네 개의 명제 (토의 정리)

1. **제약이 아키텍처를 만든다.** H800 대역폭 제한 → MLA + DualPipe + FP8. 풍족했다면 나오지 않았을 설계.
2. **보상은 단순할수록 강하다.** 학습된 RM보다 정규식과 컴파일러가 이겼다. 검증 가능성(verifiability)이 정교함을 이긴다.
3. **능력은 주입이 아니라 유도다.** RL은 elicitation이며, base 모델의 잠재력이 상한이다.
4. **효율은 스케일의 반대가 아니라 스케일의 조건이다.** 싸게 만들 수 있어야 여러 번 시도할 수 있고, 여러 번 시도해야 창발이 관측된다.

---

6. 자주 나오는 오개념 체크리스트

✗ 오해	✓ 정정
MLA는 GQA의 변형이다	다르다. GQA는 헤드를 **공유**, MLA는 **저랭크 압축 후 가중치 흡수**
MLA는 RoPE를 그냥 쓴다	아니다. 흡수와 충돌하므로 **RoPE 전용 경로를 분리(decoupled)**
V3는 671B를 다 쓴다	토큰당 **37B만 활성**
보조손실을 완전히 없앴다	시퀀스 단위 보조손실이 **α=0.0001**로 미량 잔존
전부 FP8로 학습했다	마스터 가중치·옵티마이저·임베딩·헤드 등은 **고정밀 유지**
MTP는 추론 때 병렬로 여러 토큰을 뱉는다	기본은 **폐기**. 남기면 **speculative decoding**용
R1-Zero가 R1의 이전 버전이다	아니다. **별도 산출물**이자 "SFT 없이 가능한가"의 **증명 실험**
GRPO는 PPO에 그룹을 추가한 것	핵심은 **크리틱 제거**. 그룹 통계가 가치함수를 대체
총 학습비가 557만 달러다	**최종 런의 GPU 대여료**만. 실패 실험·인건비 제외
증류 모델은 원본의 열화판이다	32B 증류판이 **동일 크기 직접 RL판을 압도**

오늘의 실험

Peer Review 30분 전 — 회고 정리

잘 된 점

- MLA의 **흡수 트릭**을 수식으로 따라가며 "왜 압축이 공짜인가"를 이해함
- GRPO를 PPO와 나란히 놓고 **크리틱 제거의 동기**를 설명할 수 있게 됨
- V3와 R1을 **되먹임 순환 구조**로 재배치하여 노드 전체 지도를 확보

아쉬운 점

- FP8 누산 정밀도 승격을 **개념으로만** 이해, 수치 검증 미완
- DualPipe 스케줄 다이어그램을 직접 그려보지 못함
- 벤치마크 수치를 외우는 데 시간을 씀 → **설계 동기 토의 시간 부족**

✓ 해결해낸 어려웠던 개념

1. **MLA 흡수(absorption)** — 압축 행렬을 인접 가중치에 미리 곱해 넣기
2. **Decoupled RoPE** — 회전 행렬과 흡수의 교환법칙 충돌
3. **보조손실 없는 부하분산** — b_i는 선택에만 개입, 게이팅 값에는 미개입
4. **GRPO 우위 계산** — 그룹 정규화가 베이스라인을 대체
5. **증류 > 직접 RL** — RL은 elicitation이라는 해석

▲ 아직 더 공부해야 할 어려운 개념

1. FP8 타일/블록 양자화와 **N_C=128 누산 승격**의 수치적 근거
2. **DualPipe** 양방향 스케줄의 버블 계산식
3. KL **k3 비편향 추정량**의 유도
4. **YaRN**의 NTK-aware 보간 원리 (scale s=40)
5. Node-limited routing의 **통신량 상한 증명**
6. MTP 순차 모듈에서 **그래디언트 흐름 차단 지점**

코드 완성 후 — 확장 실험 제안

E1. KV 캐시 계산기 + mini-MLA (난이도 ★★)

- MHA / GQA(g=8) / MQA / MLA의 토큰당 캐시량을 함수로 구현, 컨텍스트 길이별 그래프
- 소형 MLA 어텐션을 직접 구현하고 **흡수 전/후 FLOPs와 지연시간 측정**
- 검증: decoupled RoPE를 빼면 출력이 어떻게 깨지는지 관찰

E2. 보조손실 없는 부하분산 시뮬레이션 (난이도 ★★)

- 256 전문가 + top-8 토이 라우터 구성
- $\gamma \in \{0, 0.0001, 0.001, 0.01\}$ 스윕 → **전문가 부하의 지니계수/최대부하비**를 스텝별 기록
- 대조군: 기존 aux-loss 방식과 **성능-균형 트레이드오프 곡선** 비교

E3. MTP λ ablation (난이도 ★★★)

- 소형 GPT에 MTP depth 1 모듈 부착
- $\lambda \in \{0, 0.1, 0.3, 0.5\}$ → validation loss + 2번째 토큰 **수용률** 측정
- 가설: λ가 크면 수용률↑ / 본 손실↑ (논문의 0.3→0.1 스케줄 근거 재현)

E4. GRPO 그룹 크기 G 스윕 (난이도 ★★★)

- 간단한 산술 과제 + 정규식 정확도 보상으로 mini-GRPO 구현
- $G \in \{2, 4, 8, 16\}$ → 학습곡선, 스텝당 비용, **"전부 정답/전부 오답 그룹" 비율(=그래디언트 손실률)** 측정
- 확장: std 나눗셈 제거 버전과 비교 (Dr.GRPO 논의 재현)

E5. "Aha Moment" 정량 탐지 (난이도 ★★)

- R1-Distill-Qwen-1.5B로 AIME/MATH 문제 풀이 생성
- CoT 내 **재고 마커**("wait", "잠깐", "다시 보면", "however") 빈도 추출
- 마커 빈도 vs 정답률 상관 분석 → **반성이 실제로 성능과 연결되는가** 검증

E6. 보상 항목 ablation (난이도 ★★★)

- 정확도 보상만 / 형식 보상만 / 둘 다 → 학습 안정성과 출력 가독성 비교
- 예상: 형식 보상 제거 시 파싱 실패율 급증, 정확도 보상 제거 시 reward hacking

E7. 정렬세(Alignment Tax) 정량화 (난이도 ★★★)

- 언어 일관성 보상 계수를 0 → 강하게 스윕
- **성능 하락폭 vs 언어 일관성 향상폭**을 한 평면에 플롯

- → 논문이 정성적으로만 말한 트레이드오프를 **수치로 재현**

◆ E8. 루브릭 확장 실험

- 노드 기본 루브릭을 넘어: **동일 예산(GPU-hour) 하에서 "직접 RL vs 증류"** 소규모 재현
- 결론이 논문과 같은 방향으로 나오는지 확인 → 스케일 축소 시에도 성립하는가?

다음 노드 연결 지점

- **MoE 계열** → Mixtral, Qwen-MoE와 라우팅 전략 비교
- **추론 계열** → o1/o3의 test-time scaling 법칙, 그리고 "생각의 길이"가 성능과 맺는 관계
- **효율 계열** → 양자화(GPTQ/AWQ), speculative decoding, KV 캐시 압축(H2O, StreamingLLM)
- **RL 계열** → PPO → GRPO → 이후 변형들의 계보 정리

다음 회차 준비 (숙제)

담당	과제
전원	MLA 수식 손으로 유도해오기 (흡수 단계 포함)
전원	GRPO 목적함수 유도 + PPO와 차이 1장 요약
이끄미	DualPipe 스케줄 다이어그램 재현
살리미	E1·E2 스켈레톤 코드 준비
기록이	오개념 체크리스트를 팀 위키에 이관